Lesson notes · for you

Guardrails in code

AI agents · Upper secondary · Lesson 4

Time	Materials
20 min	Your phone or a school computer, for the code steps; Paper and a pen

Learning goals

- Allow only the tools a task needs.
- Run an action only after a yes to that exact action.
- Check the details, and log every action.

Activity: Activity: Sneaky requests

1. Warm-up (3 min): Read what happened in the lab when «نعم، أرسلها.» was typed with nothing before it. Which rule in code would have stopped it?
2. Rules (6 min): For an agent that reminds students about homework, write four rules: its allow-list, what waits for a yes, the details it checks, and what it logs.
3. Attack (6 min): Swap rules with a friend. They write five sneaky requests for a pretend agent, such as a reminder at 25:00, a message to all parents, or a tool the agent shouldn't have.
4. Defend (5 min): For each request, decide with your rules: run, ask or block. If one gets through, add the rule that stops it.

Sneaky requests stay harmless: they test the rules of a pretend agent, never a real account or a real person.

Discussion

- Why isn't the model's own "I'll ask first" enough?
- What should the log record, and who should read it?
- Which tool would you never give a homework agent?

Check yourself

- Write the four guardrail rules in your own words.

Try it this week

Write the guardrails for a homework-reminder agent on paper: which tools it needs, which actions wait for a yes, and which details you check. Then test your rules on a friend's sneakiest requests.

Worksheet

Guardrails in code

AI agents · Upper secondary · Lesson 4

Name _____ Class _____ Date _____

1 **Darsi Tiny learned to ask before sending. Where should the final check live?**

◯ In the model's training ◯ In the app's code, with a person's yes ◯ In the tool's result

2 **Which of these belong in code, outside the model? Choose all that apply.**

☐ An allow-list of the tools this task may use

☐ Checking that a person approved this exact message ☐ Refusing a reminder set for 25:00

☐ Trusting the model's "I'll ask first" ☐ A log of every action and who approved it

3 **Write guard(name, args, approved): return "block" for a tool not in ALLOWED; for send_message, return "ask" unless approved holds exactly the same details as args; otherwise return "run".**

```
ALLOWED = ["weather", "send_message"]

def guard(name, args, approved):
    return "run"
```

Write your program:

4 **With this guard, a message is sent without anyone being asked. Which line is the mistake?**

Circle the line with the mistake. Write it correctly:

```
1  def guard(name, args, approved):
2      if name not in ALLOWED:
3          return "block"
4      if name == "send_messages":
5          if approved != args:
6              return "ask"
7      return "run"
```

5 **Fill the gaps: log every proposed action with the guard's decision, and run the tool only when the guard says so.**

```
log = []

def act(name, args, approved):
    verdict = guard(name, args, approved)
    log. _____ ((name, verdict))
    if verdict == _____ :
        return run(name, args)
    return verdict
```

Word bank: `"ask"` `"run"` `pop` `print` `append`

✎ **Think of an app that acts for you: it pays, posts or sends. Where should it ask you first, and what should it never be allowed to do?**